Table of Contents

docFx Playground ---- 2
- Alerts ---- 5
- Code ---- 7

AllowPreview SQL ---- 8

MCP-Server (Preview) ---- 9
- Tools ---- 13

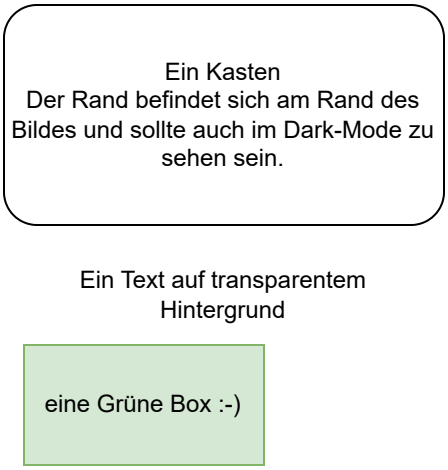
Playground

Diese versteckte Seite demonstriert ein paar Beispiele, was die Doku alles kann.

Beim Update auf eine neue docfx-Version können so ganz schnell alle Features getestet werden.

Images

Einbinden eines SVG-Bildes, welches mit Drawio erstellt wurde:



Fussnoten [1]

Und wenn man mal mehr Punkte hat, dann kann man auch sprechende Namen [2] verwenden. Es können auch ganz normal Nummern [3] verwendet werden.

Die Fussnoten werden an das Ende der jeweiligen Seite gelistet. Man gelangt mit einem Klick auf die Note dorthin.

Überschriften

Ebene 1

hier steht ein Text

Ebene 2

hier steht ein Text

Ebene 3

hier steht ein Text

Ebene 4

Die Überschrift der Ebene 4 kann auch frei verwendet werden - wie z.B. in den Release-Listen.

Ebene 5

Es gibt die Ebene 5, diese wird aber so gut wie nie verwendet.

Listen

Listen mit Boppeln am Anfang.

- Ebene 1
- Ebene 1
  - Ebene 2
  - Ebene 2
    - Ebene 3
    - Ebene 3
      - Ebene 4
      - Ebene 4
        - Ebene 5
        - Ebene 5

Man kann das auch mit Nummern schreiben.

1. Ebene 1
2. Ebene 1
   1. Ebene 2
   2. Ebene 2
      1. Ebene 3
      2. Ebene 3
         1. Ebene 4
         2. Ebene 4
            1. Ebene 5
            2. Ebene 5

---

1. können theoretisch auch in der Überschrift platziert werden. Die Nummer taucht aber rechts in der Topic-Liste auf :-(. ↩️

2. heiß eigentlich `[^blub]` wird aber automatisch nummeriert. ↩️

3. Das ist im Code die Nummer `[^2]`. ↩️

Alerts

Wir verwenden verschiedene Alert-Boxen, um wichtige Inhalte hervorzuheben.

TIP

TIP

Optional information to help a user be more successful

NOTE

NOTE

Information the user should notice even if skimming

IMPORTANT

IMPORTANT

Essential information required for user success

WARNING

WARNING

Dangerous certain consequences of an action

CAUTION

CAUTION

Negative potential consequences of an action

In Liste eingebettet

Hier ist ein Beispiel für eine Release-Liste. Dort werden z.B. NOTEs eingebaut.

- 0815: **IDE** - Es gab einen Fehler.
  und eine 2. Zeile

  NOTE

  Diese Box in eingerückt. Sie sollte hier korrekt platziert sein.

  - Innerhalb einer Box gehen auch Listen
  - wie man hier sieht.

- noch ein Punkt

TIP

Auch ein Code-Block kann in so einer Box eingebettet sein:

```
// Ein String
string s = "Hallo";
```

Code

Im Text `inline` Code. Platziert als `Link` auf eine andere Seite.

C-Sharp

```
public static class Extensions
{
    /// <summary>
    /// Gibt zurück, ob die PropertyChangedEventArgs zum übergebenen Property passen.
    /// Das ist der Fall, wenn der PropertyName identisch ist oder wenn in den EventArgs
kein Property-Name angegeben ist.
    /// </summary>
    /// <param name="propertyName">Der zu prüfende Propery-Name.</param>
    [DocfxBrowsable]
    public static bool IsProperty(this PropertyChangedEventArgs args, string propertyName)
    {
        // Ein Kommentar im Code
        return string.IsNullOrEmpty(args?.PropertyName)
            || args.PropertyName == propertyName;
    }
}
```

Sql

```
-- Sql-Server:
UPDATE dbRun_MLStrings SET ML_SearchText = LEFT(ML_Text, 200);
-- Oracle:
UPDATE dbRun_MLStrings SET ML_SearchText = SUBSTR(ML_Text, 1, 200);
```

Preview SQL

Um sich an einem Package mit der Preview Version von Framework Studio anmelden zu können, muss folgendes Skript im Framework Studio Repository ausgeführt werden:

Spalte hinzufügen

```
ALTER TABLE tblRep_Label ADD Lbl_AllowPreview char(1) NULL;
```

AllowPreview Wert setzen

Allgemeines SQL Skript

```
UPDATE tblRep_Label
SET Lbl_AllowPreview = 1
WHERE Lbl_ID = 'VERSION ID aus PackageManager'
```

SQL Skript für FSDemo und FSDemoCustomize Packages

```
UPDATE tblRep_Label
SET Lbl_AllowPreview = 1
WHERE Lbl_P_ID IN ('9a7c2d2d05e54ae6a97a9f2b919d6648', 'fde6193cb96842aaaf0d5e005b1a0c8d')
```

MCP-Server (Preview)

Der MCP-Server stellt eine Schnittstelle bereit, über die KI-Assistenten direkt auf das FrameworkStudio-Repository zugreifen können. Er implementiert das Model Context Protocol (MCP) und läuft als AddIn innerhalb der FrameworkStudio IDE.

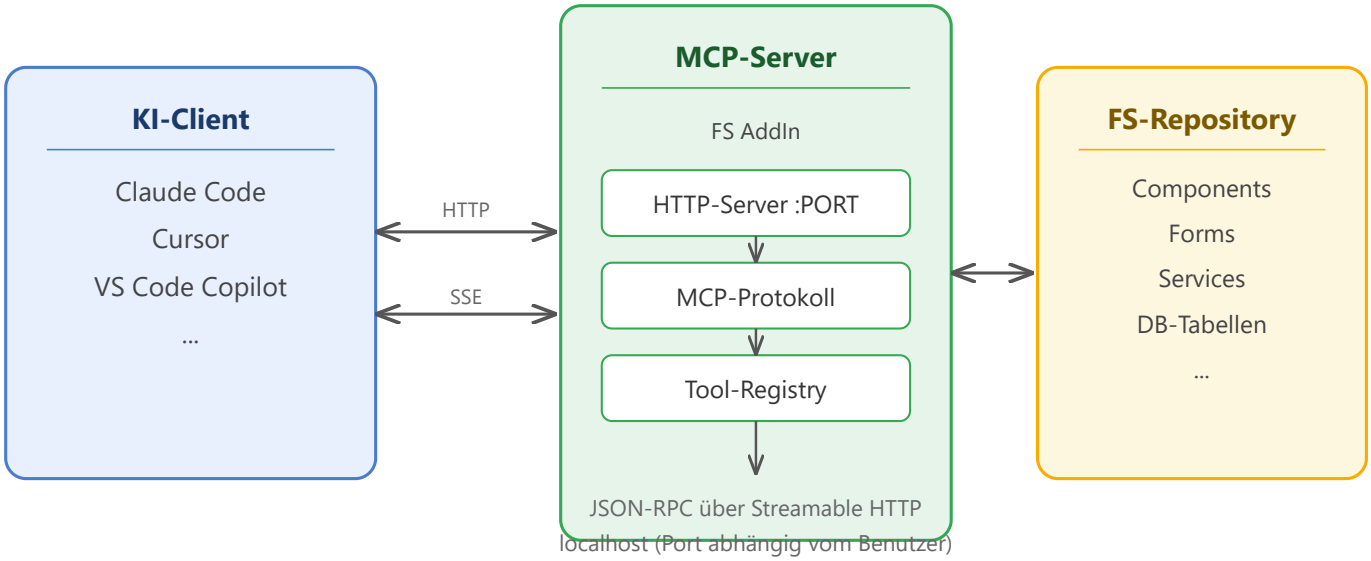
> **IMPORTANT**
>
> Der MCP-Server ist ein **Preview-Feature** in Version 4.8. Funktionsumfang und Schnittstellen können sich in zukünftigen Versionen ändern.

Was ermöglicht der MCP-Server?

KI-Assistenten wie Claude Code, Cursor oder GitHub Copilot können über den MCP-Server:

- **Elemente auflisten und lesen** — Components, Forms, Services, DB-Tabellen und alle weiteren FS-Elementtypen
- **Code inspizieren** — Methoden-Quellcode abrufen
- **Suchen** — nach Namen, im Code, nach Abhängigkeiten und in der Element-Dokumentation
- **Änderungshistorie abfragen** — Versionsverlauf von Elementen, Methoden und CheckIns

Architektur



Der MCP-Server wird über das **Tools-Menü** in der IDE gestartet und läuft als HTTP-Server auf `localhost`. KI-Clients kommunizieren über das standardisierte MCP-Protokoll (JSON-RPC über Streamable HTTP mit Server-Sent Events).

Die Tool-Registry stellt alle verfügbaren Operationen bereit. Jedes Tool greift über die FS-API auf das Repository zu — im Kontext des eingeloggten Benutzers.

Port-Ermittlung

Der Port wird **automatisch** aus dem Windows-Benutzernamen abgeleitet:

1. Aus dem Benutzernamen (lowercase) wird ein Hashwert berechnet
2. Der Hashwert bestimmt den Port im Bereich **30100–35099**
3. Falls der ermittelte Port belegt ist, wird der nächste freie Port verwendet (bis zu 10 Versuche)

Dadurch erhält jeder Benutzer auf einer Maschine einen eigenen Port, ohne manuelle Konfiguration.

> **TIP**
>
> Der tatsächlich verwendete Port wird beim Start im **Output-Fenster** der IDE angezeigt.

Einrichtung eines KI-Clients

Für die Arbeit mit dem MCP-Server steht das Repository **trade-fs-claude-workspace** als Einstiegspunkt bereit. Es enthält vorkonfigurierte Skills, Domänenwissen und Setup-Skripte für Claude Code und GitHub Copilot.

Kurzanleitung

1. **Repository klonen** — `trade-fs-claude-workspace` aus der internen Organisation herunterladen
2. **FrameworkStudio starten** und Projekt laden
3. **MCP-Server aktivieren** — Menü: **Tools** → **MCP-Server starten**
4. **KI-Client starten** — der MCP-Server-Endpoint ist `http://localhost:PORT/mcp` (PORT aus dem Output-Fenster übernehmen)

> **NOTE**
>
> Die MCP-Konfiguration im Workspace-Repository enthält einen Standardport. Da der tatsächliche Port vom Benutzernamen abhängt, muss die URL ggf. einmalig angepasst werden.

Voraussetzungen

- FrameworkStudio 4.8 oder höher
- Aktive FS-Session mit eingeloggtem Benutzer
- Ein MCP-fähiger KI-Client (z.B. Claude Code, Cursor, VS Code mit Copilot)

Einschränkungen

- Der Server ist nur lokal erreichbar (`localhost`) — kein Zugriff von außen
- Pro IDE-Instanz läuft maximal ein MCP-Server
- Der Server arbeitet im Kontext des eingeloggten FS-Benutzers — Berechtigungen gelten wie in der IDE
- In der Preview-Version stehen ausschließlich **lesende** Operationen zur Verfügung

Bekannte Limitierungen (Preview)

Nicht unterstützte Elementtypen

DocumentationOwner wird von keinem Tool unterstützt:

- Kann nicht aufgelistet werden (`fs_list`)
- Kann nicht nach Namen gesucht werden (`fs_search_names`)
- Kann nicht abgerufen werden (`fs_get`)
- `fs_search_documentation` findet zwar den *Inhalt* von DocumentationOwner-Dokumentation, aber man kann einen DocumentationOwner nicht nach seinem Namen finden

AccessUnit und **Datasource** können über `fs_list` aufgelistet, aber nicht über `fs_search_names` nach Namen gesucht werden.

Lücken bei bestehenden Tools

Bereich	Limitation
DB-Tabellen-Historie	`fs_get_element_history` findet keine DB-Tabellen, da diese über Datasources statt über Namespaces organisiert sind.
Code-Suche	`fs_search_code` durchsucht nur Component-, Form-, Service- und ModularComponent-Methoden. Code in GlobalEvent- und ServiceContract-Methoden wird nicht indiziert.
Dokumentationssuche	`fs_search_documentation` durchsucht Property-MLDoc, Form-Doku und DocumentationOwner-Doku — aber nicht die MLDoc direkt auf Component-Ebene.
Such-Cache	Der interne Cache für `fs_search_names` und `fs_search_code` wird nur bei einem Label-Wechsel aktualisiert. Einzelne CheckIns innerhalb desselben Labels werden erst nach einem Neustart des MCP-Servers sichtbar.

Sicherheitshinweis

Der MCP-Server verfügt über **keine Authentifizierung**. Jeder Prozess auf der lokalen Maschine mit Zugriff auf den jeweiligen `localhost`-Port erhält uneingeschränkten Lesezugriff auf alle Repository-Daten — einschließlich Methoden-Quellcode und Datenbankschema.

Das ist für ein lokales Entwickler-Tool bewusst so gestaltet. Daraus folgt:

- Den Server **nicht** auf einer anderen Adresse als `localhost` betreiben
- Den Server **beenden** wenn die IDE-Session abgeschlossen ist
- Auf gemeinsam genutzten Entwicklermaschinen beachten, dass andere lokal angemeldete Benutzer ebenfalls Zugriff haben

Tools

Alle Tools des MCP-Servers arbeiten **read-only** — es werden keine Änderungen am Repository vorgenommen. Die Ergebnisse sind auf das aktive Package und dessen Abhängigkeiten beschränkt.

> **TIP**
>
> Die vollständige Parameterbeschreibung jedes Tools wird vom MCP-Server über `tools/list` bereitgestellt. KI-Clients erhalten diese automatisch bei der Verbindung.

Umgebung

Tool	Beschreibung
`fs_get_environment`	Aktives Package, Domain, Verbindungsinformationen

Elemente auflisten

Tool	Beschreibung
`fs_list`	Alle Elemente nach Typ auflisten. Unterstützte Elementtypen: `component` , `form` , `service` , `modular_component` , `namespace` , `package` , `db_table` , `metadatatype` , `global_object` , `datasource` , `global_event` , `service_contract` , `service_host` , `workflow` , `report` , `access_unit` , `data_contract` . Parameter: `elementType` , `namespaceName` (optional), `offset` + `limit` (Pagination).

Elemente lesen

DevObjects (Component, Form, Service, Modular Component)

Tool	Beschreibung
`fs_get_devobject`	Details auf DevObjects (Component, Form, Service, Modular Component) abrufen. Parameter: `elementType`

Tool	Beschreibung
	(component|form|service|modular_component), `fullName` , `detail` (optional: summary|full, default full), `method` (optional: nur diese Methode mit vollem Code), `property` (optional: nur diese Property). **Detail-Level:** `summary` zeigt Struktur (Property-Namen/Typen, Methoden-Signaturen) ohne Code-Bodies — ideal für Orientierung großer Komponenten. `full` gibt komplettes YAML mit allen Code-Bodies (Standard).

Andere Elementtypen

Tool	Beschreibung
`fs_get`	Details auf andere FS-Elemente abrufen. Parameter: `elementType` (package|db_table|metadatatype|data_contract|global_object|global_event|service_contract|se `name` .

Suche

Tool	Beschreibung
`fs_search_names`	Element-Namen und FullNames durchsuchen (alle Elementtypen)
`fs_search_code`	Volltextsuche im Methoden-InnerCode über Components, Forms und Services. Parameter: `pattern` , `regex` (optional: false für Literal-Match, true für Regex-Pattern).
`fs_search_dependencies`	Strukturelle Abhängigkeiten eines Elements finden (Properties, Typen, Vererbung)
`fs_search_documentation`	MLDoc-Felder auf Metadatatypes, Properties und Forms durchsuchen

NOTE

`fs_search_code` durchsucht nur den Methoden-InnerCode, nicht Declarations oder Descriptions. Indiziert werden nur Component-, Form-, Service- und ModularComponent-Methoden — **nicht** GlobalEvent- oder ServiceContract-Methoden.

`fs_search_names` durchsucht Element-Namen, nicht Property- oder Methodennamen innerhalb eines Elements. AccessUnit, Datasource und DocumentationOwner sind nicht in der Namenssuche enthalten.

`fs_search_documentation` durchsucht MLDoc-Felder auf Metadatatypes, Component-Properties, Forms und DocumentationOwnern. Die Suche findet Doku-**Inhalte** — eine Suche nach dem **Namen** eines DocumentationOwners ist nicht möglich.

Änderungshistorie & Vererbung

Tool	Beschreibung
`fs_get_element_history`	Versionsverlauf eines Elements (Datum, Benutzer, Kommentar)
`fs_get_method_history`	Code-Änderungen einer Methode über verschiedene Versionen
`fs_get_checkin_history`	Globaler CheckIn-Verlauf (wer hat wann was eingecheckt)
`fs_get_inheritance_tree`	Vererbungsbaum eines Elements (alle Parent und Child Klassen)

Hinweise zur Nutzung

- Für große Components (50+ Properties, 30+ Methoden) ist `detail=summary` effizienter als `detail=full`. Summary zeigt die Struktur ohne Code-Bodies.
- Mit dem `method=MethodName` Parameter können Sie nur eine einzelne Methode mit vollem Code laden statt die ganze Component.
- Mit dem `property=PropName` Parameter können Sie nur eine einzelne Property auslesen statt die ganze Component.
- **Collections** sind Components mit `ComponentType=Collection` — sie erscheinen in `fs_list` mit `elementType=component`, nicht in einer separaten Liste.
- `fs_search_dependencies` zeigt was ein Element **referenziert** — nicht welche Elemente das Element referenzieren. Für die Gegenrichtung `fs_search_code` verwenden.

- `fs_get_element_history` unterstützt keine DB-Tabellen — nur Namespace-basierte Elemente (Component, Form, Service, etc.).
- Der Such-Cache (`fs_search_names` , `fs_search_code`) wird nur bei Label-Wechsel aktualisiert. Nach einem CheckIn innerhalb desselben Labels den MCP-Server neu starten, um aktuelle Daten zu erhalten.